P. Akhil

CDS/2025/0788 Data Structures

1) Reverse a string unsing stack?

Algorithm

(i) Push each character of the string onto the stack

(ii) Pop charactor one by one and store them back into the string.

(iii) Point the reversed string.

Code

```
# include <stdio.h>
# include <string.h>
# define max 100
  // stack structure
    char stack [max];
    int top = -1;
  // Push character into stack
    void push (char c) {
        stack [++ TOP] = c;
    }
  // POP character from start
    char pop ( ~~char c~~ () {
        return stack [TOP - - ];
    }
    int main {
        char str [max];
        Printf ("enter a string ");
        scanf ("%s, str);
        int len = str len(str);
  // Push all characters onto stack
    for (int i = 0; i < len ; i ++) {
```

```
    Push (str[i]);
  }
// Pop charactar to release
  for (int = 0; i<len; i++) {
    str[i] = pop();
  }
  Printf ("reverse string: str);
  return 0;
}
```

Output :-

Enter a string : ~~Data~~ DATA

Reverse a string: ~~ATAD~~ ATAD

2) Balanced Parenthess:

Algorithm:-

```
// scan each charactor of the expression.
# include <stdio.h>
# include <string.h>
# define max 100
  char stack [max];
  int top = -1;
  void push (char) {
     int (top = -1)
    Printf ("Stack out flow")
  } else {
     stack (++ Top] = x;
  }
  }
  char pop () {
```

```
if (top = -1) {
    Printf ("stack under flow \n");
    return -1;
  } else {
    return stack [TOP--];
  }
}
int balanced (char * (x)) {
    for (int i=0 ; i<strlen (ex); i++ ) {
        char ch = ex i[i];
        if (ch= ='(' || ch= = '{' || ch= = "[" -) {
          Push (ch);
    } else if (ch= = ')' || ch = "}" || ch= = "]" ) {
          if (top = = -1) {
              return 0;
        } Chai popped = POP();

    if (ch= = ")" && popped != '(' || ch= = '}' && popped = {
     ch = = ']' && Popped != '(4')) {
      return 0;
    }
  }
  return (top = = -1);
}
int main() {
 Char exp [max];
 print f ("enter an expanstion ");
 scanf (" % s, &exp) &
```

```
if (balanced(exp)) {
    Print ("Balanced expression");
} else {
        print (" not balanced expression).
    }
    return 0,
}
```

Input = $(A+B)^{+}(1-P)$

output = Balanced expression

3) Next gertel element

Algorithem

1) Transverse array from right to left
2) For each element, pop, stake, element that are

   C = current element
3) If stack is empty NGE = -1; else NGE = top of stack
4) Push current to stack

Code;

```
#include <stdio.h>
# define max 100
int stack [max];
int top = -1;
void push (int val) {
stack [++TOP] = val;
}
int pop () {
    return Stack [TOP - -];
}
```

```
int is empty () {
    return top = = -1;
    }
int peek () {
    return stack [Top];
    }
int main () {
    int arr [max , nge [max], n;
Printf ("enter number of element");
scanf (" %d, & any[i] );
  }
// Transverse from right to left
  for (int i= n-1; i>0; i--){
// pop element smaller or equal to current.
while (! is empty() & & peak ( ) < = arr [i]) {
    pop (i);
// if stack empty, no greater element exist

  n g c [i] = is empty () -1 ; peek ;
  push (any [i]; // push current element
// print result.
  printf ("in element -> next greatest element \n")
  for int i= 0; i<n; i++ ){
  printf ("%d" -> %d \n"), arr [i], ngc [i]);
```

return 0;

}

output:

enter number of element : 5

enter element : 4 5 2 10 8

element -> next greatest element

4 -> 5

5 -> 10

2 -> 10

10 -> -1

8 -> -1

4) print a queue simultation

Algorithm:

① Intilizae front = -1, rear = -1, size = 0

② Add document (enque):

a) If queue is full, print " queue full "

b) else if empty, set front = rear = 0

c) else increment rear

d) store document name at quece [rear]; increm

③ print document (Dequequl)

a) If quece is empty, print " No pending documen

b) else print document cut all [front]

c) If front == rear, rest front = rear = -1

d) else increment front, dcement size.

(4) Display pending

(a) If empty print "ques is empty

(b) Else print all documents from front to rear

Program:-

```
# include <stdio.h>
# include <string.h>
// maximum number of documents in the queue
# define max 10
# DOC-LEN 50
// Queue structure for printer.
char queue [max] [DOC-LEN];
int front = -1, rear, size of
// check if queue is full.
int isfull() {
    return size == max; }
// check if queue is empty
int isempty() {
    return size == 0; }
// check document to the queue
viod add document (char doc name)() {
  if (isFull ()) {
    Printf("Printer queue is Full! Cannot add 's'
                                doc name);
  return;
}
```

```
if (is empty()){
    front rear = 0; }
  Else {
    rear (rear 1)% max;}
    strcpy queue [rear], doc name);
    size ++;
//printf (remove) the front document
  viod print document () {
    if (is empty ()){
      printf ("no pending documents in the queue\n);
      return n;
    }
    printf ("pointing document documents in the queue\n);
      return;
    if (front == rear){
      front = rear = -1;}
  else {
    front = (front+1 % max;}
    size--;
  }
// display all pending documents
  viod display queue () {
    if (is empty ()){
    printf ("no pending document's queue is empty\n");
    return ;
```

```
}
printf ("\n -- pending Doucments (%d -- \n", size);
    int i= front, count=0;
    while (count < size) {
      printf ("%d . %s\n", count+1, queue [i]);
      i=(i+1) % max;
      count ++; }
      Printf ("------ \n"); }
int main () {
  int choice;
  char data name [DOC_LEN];
  Printf ("===== Printer queue simulation ===\n");
do {
  printf ("\n 1. Add Document \n");
  printf ("2. print Document \n");
  printf ("3. Display pending document \n");
  printf ("4. exit \n");
  print f ("%d ; & choice : ");
   get char ();
switch (choice){
  case 1:
     print f ("enter document name : ");
       pgets (doc Name, DOC-LEN, stdin);
       doc Name [strcspn (doc Name, "\n")]= '0';
       add document (doc Name);
       break;
```

```
case 2:
    print document();
    break;
case 3;
    display queue ();
    break;
case 4;
    printf ("existing printer queue good bye! \n");
    break;
deaufault:
    printf ("invalid choice. Try again \n"); }
} while (choice != 4);
return 0; }
```

output:

```
===== Printer Queue simulation =====
1. Add Document
2. print Document
3. display Pending Documents
4. Exit
Enter choice: 1
Enter document Name: Report pdf
document Report .pdf 'added to the printer queue
```

(5) circular queue basic operations

(1) Initialize front = -1, rear = -1

(2) enque (value);

(a) If full ((rear++) % max = front), print "overflow"

(b) else if empty, set front = rear = 0

(c) else rear = (rear + 1) % max

(d) queue [rear] = value

(3) Dequeue:

(a) If empty (front == -1), print "Queue empty

(b) store queue [front] as removed value

(c) if front = rear (single element), reset front = rear = -1

(d) else front = (front + 1) % max

(4) Peek:

(a) If empty, print "queue empty"

(b) else print queue [front]

(5) Display:

(a) If empty, print "queue empty"

(b) else print queue [front]

~~(5) Display:~~

a. program:

```
# include <stdio.h>
# define 'max' 6
int (queue [max];
int front = -1, rear = -1;
```

```
int is empty (){
        return (front = -1); }
// insert element into circular queue
viod enque (int value){
    if (is full ()){
     printf ("circular queue is full ! Cannot inserted\n", value)
    return ;
    }
  if (is empty ()){
      front = rear = 0.
    }
  else {
    rear = (rear + 1) % max ; }
  queue [rear] = value;
  printf ("enqued %d\n", value); }
// remove element from circular queque
viod dequeue () {
      if (is empty ()) {
      printf ("circular queue is empty! nothing to remove\n")
    return ; }
printf ("dequed %d\n", queue (front));
  if (front = = rear) {
      front = rear = -1 }
  else {
    front = (front +) % max ; } }
```

```
// display all element in the circular queue
void display() {
    if (is empty()) {
    printf ("circular queue is empty) no front element");
    return;
  printf ("front element (peek) %d \n", (queue [front]);
// display all element in the circular queue
void display () {
    if (is empty ()) {
    Print f ("circular queue is empty \n");
    return; }
  printf ("\n -- circular queue elements --- \n");
  print f ("front ");
  int i = front;
while (1) {
    printf ("%d ", queue [i]);
    if (i == rear) break;
    if (r == rear) break;
    i = (i+1) % max; }
  print f ("← rear \n");
  print f ("---- \n"); }
int main() {
  int choice, value;
  printf ("\n === circular queue operation ===\n");
```

```
do {
    printf ("\n 1- Enqueue \n");
    printf ("2. Deque \n");
    printf ("3. Peek \n);
    printf ("4. display \n");
    printf ("5- Exit \n");
    printf ("enter choice");
    scanf ("%d", &choice);
switch (choice) {
    case 1:
        printf ("enter value to insert");
        scanf ("%d", &value);
        enque (value);
        break;
    case 2:
        deque ();
        break;
    case 3:
        peek ();
        break;
case 4:
        display ();
        break;
default:
        printf ("Invalid choice \ Try again \n"); }
while (choice != 5); return 0; }
```

Sample output:

=== Circular Queue operations ===

1. enque 2. Dequeue 3. peek 4. Display 5. Exit

Enter choice: 1

enter to insert: 10

enqued : 10

enter choice: 1

enter value to insert: 30

enqued: 30

enter choice: 4

.... circular queue elements ...

front → 10 30 ← rear